

Migration status

Updated: 2026-09-21 · **Phase:** step 4 — **under way**. Four architecture projects exist; eleven board configurations build.

This file is the cold-start entry point. Read it, then `docs/specs/2026-09-20-micro-os-plus-iii-smp-unification-design.md` for the full design and the measurements behind it.

Where things stand

Steps 1 and 2 are complete. Six repositories exist; both A-profile architecture projects build all 24 of their targets from a single copy of every test, and all twelve tests pass on a Raspberry Pi Zero 2 W on **both** ports — each measured on that port, neither inferred from the other.

Step 3 is in progress. The AArch32 port now carries a second board, the Luckfox Lyra B (RK3506, 3× Cortex-A7): 19 targets build, and both ports still pass their QEMU suites (11/1/0 each) after the restructure it required. What that second board changed, and the seven portability defects it exposed in shared code, are written up in `aarch32-second-board.md`.

The Lyra is a hardware-only board, decided and closed. QEMU models none of the RK3506's own blocks, so an emulated Lyra would be a generic Cortex-A7 wearing the name — and the DWC2 gadget, the SD host and the Cortex-M0 mailbox, which are what make this board worth having, are exactly the parts not modelled. Its `board.cmake` sets no `UOS_BOARD_LINKER_QEMU`, so it builds one image per test rather than two, and `test/qemu.sh` says so if asked. This is not a gap to close later; do not reopen it.

Step 4 is under way. `micro-os-plus-iii-cortexm` exists, with six boards: three STM32F4 at `OS_NCPU=1` and three RP2350 boards at `OS_NCPU=2` — the Raspberry Pi Pico 2, the WeAct RP2350B with 8 MB of PSRAM, and the Pi-Zero RP2350B. The STM32 boards are the first `OS_NCPU=1` boards in the workspace, so they are also the first to build the kernel's **non-SMP** branch — every board before them had more than one CPU. Nothing on this port has run on hardware yet. The port, its six boards and where its SMP lives are written up in `cortexm-port.md`.

TMP7/

```
|─ micro-os-plus-iii-smp/      kernel + shared build rules + test runners
|   |─ src/ include/         the kernel, upstream path-for-path
|   |─ port/smp-common/      os-decls.h, shared by both ARM ports
|   |─ test_smpl/           the two runners -- see test-smpl.md
|   |   |─ run-qemu.sh       the shared QEMU suite runner
|   |   └─ run-hw.sh        the shared hardware session runner
|   |─ cmake/               toolchains + uos_add_app
|   └─ tools/verify-kernel-compiles.sh
|─ micro-os-plus-iii-devices/  flatfs, FatFs; SD per SoC (BCM2837, RK3506);
|                               silicon support per SoC (BCM2837, STM32F4,
|                               RP2350)
|─ micro-os-plus-iii-aarch64/  ARMv8-A port
|   |─ src/ include/         ARMv8-A, every board
|   └─ test/                 everything board- or test-specific
|       |─ CMakeLists.txt    one loop over test/<board>/*/
|       |─ hw.sh  qemu.sh    dispatchers; BOARD picks the directory
|       |─ boards/{rpi-zero-2w,rpi3b}/ board.cmake, include/, src/,
|       |                               linker, OpenOCD, hw.sh, qemu.sh
|       |─ rpi-zero-2w/ rpi3b/ each board's test applications
|       └─ build*/           the build trees
|─ micro-os-plus-iii-aarch32/  ARMv7-A port
|   |─ src/ include/         ARMv7-A, every board – incl. the scheduler
|   └─ test/                 same shape
|       |─ boards/{rpi-zero-2w,rpi3b,luckfox-lyra}/ BCM2837 · BCM2837 · RK3506
|       └─ rpi-zero-2w/ rpi3b/ luckfox-lyra/      12 · 12 · 19 applications
|─ micro-os-plus-iii-cortexm/  Cortex-M port – M4F and M33, 1 and 2 CPUs
|   |─ src/ include/         upstream's single-core core (STM32 boards)
|   |─ src/rtos/os-core-rp2350.cpp
|   |   include-rp2350/      the same core plus an SMP branch (pico2)
|   └─ test/                 same shape
|       |─ boards/{nucleof411,weactf411,weactf412}/
|       |                               STM32F411RE · F411CE · F412RE
|       |─ boards/{pico2,pico2-rp2350b-psram,pico2-pizero}/
|       |                               RP2350 4 MB · RP2350B +PSRAM · RP2350B
|       |                               the last two include pico2's board.cmake
|       └─ <board>/          1 · 2 · 2 · 12 · 14 · 10 applications
|─ micro-os-plus-iii-smp-old/  READ ONLY – the migration source
```

Outside `src/` and `include/` – which are the ISA and nothing else – an architecture project is all `test/`: the boards, their tests and their build trees. How that is laid out and how to run it is

```
[`tests-in-aarch32-aarch64.md`](tests-in-aarch32-aarch64.md).
```

Migration source (read-only)	<code>TMP7/micro-os-plus-iii-smp-old</code>
Remotes	<code>GIT/micro-os-plus-iii-</code> <code>{smp,devices,aarch32,aarch64,cortexm}.git</code>
Old remote	<code>GIT/micro-os-plus-iii-smp-old.git</code>

An architecture project holds **no copy** of the kernel or the devices repo. CMake resolves both as sibling directories, overridable with `-DUOS_SMP_DIR=` and `-DUOS_DEVICES_DIR=`. Whoever builds one clones what it needs. One working copy of the kernel serves every architecture project on the machine, so an edit to it is visible to all of them at once. The dependency runs one way: neither the kernel nor the devices repo knows an architecture project exists.

Done

1. **Remotes reorganised**, old tree analysed and measured, design spec written. Every figure in the spec came from the tree, not estimation.
2. **Step 1 — skeleton and kernel**. Kernel vendored once, flattened to the repository root so it matches upstream path-for-path. Three toolchains share one preamble. `uos_add_app` replaces 214 Makefiles.
3. **Devices split out** (`micro-os-plus-iii-devices`). All seven sources compile for armv7-a and armv8-a from one copy; ~12,000 duplicated lines collapse to one copy.
4. **Step 2a — the twelve test applications unified**. 15,507 lines of application C++ become 7,610. Nothing ISA-specific is left in a test.
5. **Step 2b — both ARM architecture projects**. Each builds every board's tests from one loop over `test/${BOARD}/*/`, in two variants. No test name and no board name appears in either port's build files. 24 targets per Pi board. The Lyra is 19: it is hardware-only, so it builds one image per test rather than two.
6. **QEMU suites green**. `11 passed, 1 skipped, 0 failed` on both, with gcc 15 and one suite at a time. `usb_test` skips by design — QEMU emulates no USB device mode — as the predecessor suite also recorded.
7. **Documentation**. `docs/smp-construction.md` (the port contract, the division of labour, and the defines per folder) and `docs/building-aarch32-aarch64.md`, both with PDFs, rendered by a single `docs/md2pdf.py` that replaces three near-identical copies.
8. **Hardware green on both ports**. All twelve pass on a Zero 2 W over a J-Link, `usb_test` included — the one QEMU can never run. `test_smpl/run-hw.sh` plus a ~40-line `test/hw.sh` per port replace the predecessor's 48 per-test runner scripts.

What hardware found that QEMU could not

Five real defects, all in paths the emulator does not reach:

Defect	Fix
Every blinking test drove GPIO16 , so nothing lit	<code>LED_PIN</code> is a board fact now, 29 on the Zero 2 W, set once for every test. Only <code>usb_test</code> had ever set it — and the predecessor Makefiles had the same hole.
<code>usb_test</code> answered a PUT after streaming its hexdump and listing	<code>send_reply</code> moved to immediately after the store. A completed store looked like a hang.
Console posts blocked the USB service thread, so bulk OUT was not re-armed	<code>try_send</code> , with <code>console_dropped</code> in the tally. Diagnostics must never throttle the protocol.
<code>LIST</code> compared file names case-sensitively	FatFs on a boot card has no long names and returns <code>XFER.BIN</code> ; flatfs under QEMU keeps the name as sent.
<code>kOutChunkMax</code> overflowed <code>PKTCNT</code> — a 1 MiB transfer stored 61440 bytes, silently truncated	<code>D{I,0}EPTSIZ</code> is bounded by two fields. <code>GHWCFG3 = 0x0ff000e8</code> : XFRSIZ 19 bits but PKTCNT 10 bits = 1023 packets , so 65472 bytes at full speed. The chunk derives from the packet limit now, with <code>static_assert</code> 's on both fields.

What made the test deduplication possible

The ISA seam in the applications turned out to be tiny, and mostly not a seam:

what the two copies disagreed on	resolution
<code>dsb</code> vs <code>dsb sy</code> , <code>dmb</code> vs <code>dmb ish</code>	not a difference — the explicit forms assemble on ARMv7-A too. Verified with both assemblers.
<code>cpsie i</code> vs <code>msr daifclr, #2</code>	already a kernel API: <code>interrupts::uncritical_section::enter()</code> , implemented by every port
<code>mrrc p15...c14</code> vs <code>mrs cntpct_el0</code>	already in the port's <code>timer_arm.hpp</code> ; the tests were re-reading the register themselves
<code>"(AArch64)"</code> in banners	<code>PORT_BANNER_ISA</code> / <code>PORT_BANNER_CPU</code> , beside the existing <code>PORT_BANNER_LONG</code> / <code>SHORT</code>

Two pieces of per-application boilerplate were also collected: `test-smp-boot` (idle stacks, idle body, `smp_install_boot_threads()`, in ten of twelve tests) and `test-console` (the mutex-guarded print helpers, in three). The shared boot helper generates thread names from `OS_NCPU` instead of listing four, so it no longer assumes a four-core BCM2837.

Two defects found and fixed on the way

- **The kernel exported all 63 sources as one target**, but the proven rpi builds compiled exactly 37. Linking the rest breaks the build: `posix-io/c-syscalls-posix.cpp` declares `read` / `write` returning `ssize_t`, newlib declares them returning `int`. `micro-os-plus::iii` is now that measured 37-source core; posix-io, drivers, generic startup, newlib-reent, semihosting and the three

trace backends are opt-in targets. Two entries were headers listed as sources and are gone — which is why 63 entries were always 61 compiled files.

- The Cortex-A7 port's `os-decls.h` was missing `volatile` on `lock_state[]`, which several cores read and write. The shared copy in `port/smp-common/` is the correct one, so adopting it fixes that port rather than merely deduplicating it.

Decisions — all resolved

Spec Section 11 has the table; nothing is open. Revision 2 set the shape: architecture-specific ports (not target-specific), six repositories, architecture projects independent and *outside* the main repo, devices in their own repo, `cortexm` and `posix-arch` both to become SMP.

Next step

Step 2 is done. Gate: all 24 targets build (**met, both ports**); QEMU suites pass (**met, 11/1/0 both ports**); hardware tests pass on the Pi (**met, 12/12 both ports**); no device or test source exists in more than one repo (**met**).

Step 3 (`aarch32` gains the RK3506) is **under way**. Gate: `exception_handler.cpp` shared unmodified by both boards (**met**); the board builds (**met, 19 targets**); neither existing port regressed (**met, QEMU 11/1/0 on each**). Open: running the RK3506-specific tests, which are hardware only — the six carried over from the predecessor (the USB gadget pair, the SD pair, the interrupt pair and the Cortex-M0) have never been run, and `smp_test4` is reported not to work.

Emulating the Lyra is **not** open. It was considered and closed: see above.

Step 4 (`cortexm`) is **under way**. Gate: no app source exists more than once; STM32 boards pass at `OS_NCPU=1`; pico2 passes at `OS_NCPU=2`. The repository exists and all six boards build — three STM32F4 (5 applications) and three RP2350 (36 images). **Nothing on this port has been run on hardware**, so the two `pass` clauses of the gate are not met and are not claimed. What the ELF files show: the single-core tests carry no `launch_core1` and no per-core idle table while the SMP ones carry both, the PSRAM tests put `.bss` and the heap at `0x11000000` while their board-mates keep them in internal SRAM, each USB image carries only its own class driver, and the two kernel-less probes carry zero `os::rtos` symbols where their board-mates carry 145.

The spec framed this step as "merge pico2's SMP core with the STM32 boards". Measurement changed the shape, for the better:

- pico2's core is **not portable Cortex-M SMP**. Its kernel lock is the RP2350's SIO hardware spinlock 0, its IPI the SIO inter-core FIFO on IRQ 25, its CPU index `SIO_CPUID`. All three are silicon, so in this layout they are **board** code — and `cortexm` gained SMP by a board arriving, exactly as the Lyra's GIC-400 SGI and the Pi's are board code.
- The four "pico2 variants" differ in **one** thing, and it is that same kernel lock: `pico2` uses the SIO spinlock, `pico2-std` Peterson's algorithm in software (the RP2350 has no global exclusive monitor,

so LDREX/STREX does not work across the cores), `pico2-sdk` the Pico SDK — and `pico2-sdk-min` has **byte-identical** port files to `pico2-sdk`, so it is not a separate port at all. Only `pico2` is carried so far.

- The apps are one set copied four times, 99.0–99.5% identical.

Then: step 5 (`posix-arch` — a new SMP implementation, one host thread per CPU).

Things a fresh session should not rediscover

- **Two RP2350 tests compile no kernel**, and that is the point of them: `smp-test0` (dual-core bring-up) and `exc-test` (first-exception probe) run before a scheduler exists. `uos_add_app()` takes `NO_KERNEL`, the port exports a board-only `micro-os-plus::cortexm-bare` beside the full target, and `BOARD_TEST_NO_KERNEL` names them. Do not rewrite them as RTOS tests.
- **On the RP2350 boards, SMP is a per-TEST fact, not a board fact.** `smp-test1` (single-core RTOS bring-up) and `sc-test-ko` (single-core kernel-object test) define `OS_USE_SMP_SCHEDULER` in no Makefile. `uos_add_app()` derives that define from `NCPU`, and `board_test_ncpu()` names the two tests. `sc-test-ko` is the only test on that silicon exercising the kernel's **non-SMP** branch.
- **The nested-interrupt tests need `-g0 -mlong-calls` on `main.cpp` alone.** They place handlers in `.data` to run from RAM instead of XIP flash. Without those two options the assembler stops with `Error: leb128 operand is an undefined symbol`. A call from flash to a `.data` function is also out of a Thumb `BL`'s reach.
- **`pico2-pizero` builds a different port branch.** It forces `__ARM_ARCH_8M_MAIN__` where the other two RP2350 boards force `__ARM_ARCH_7EM__`. The M33 is ARMv8-M Mainline and the predecessor's Makefiles for that board said so; the two branches have never been compared on the desk.
- **TinyUSB is carried once.** The predecessor kept `src/usb-cdc/` and `src/usb-hid/` byte-identical apart from one class driver each. The per-test include directory must come first — TinyUSB includes `tusb_config.h` by plain name.
- **`smp-test-mini-a-usb-cdc-acm_agy` is a byte-identical duplicate** of its sibling, `main.cpp` and `Makefile` both. Like `pico2-sdk-min`, it is not a separate thing.
- The four patched kernel copies are **byte-identical** (`diff -rq`).
- The kernel differs from pristine upstream in exactly **6 files**, all under `rtos/`. That is the entire SMP delta.
- **Upstream fork point**, preserved because the spec drops `BASE`: `micro-os-plus-iii @ c47f806b57f8b9b870ec7b89ded663dec354df96` (branch `xpack-development`), `micro-os-plus-iii-cortexm @ 687e975caa298519cb214d4b5a9774c48f784816`.
- **`src/rtos/os-core.cpp` exists twice on purpose.** The kernel's is the portable SMP scheduler; each port has its own at the same relative path with the bring-up, the kernel lock and the context-switch hooks. Both are compiled, exactly as the old Makefiles did. They do not collide.
- **A port needs its machine flags at link time too**, not just compile time. Without them the driver picks the wrong multilib and every AArch32 link fails with "uses VFP register arguments".

- **Rebuild the port you are about to test.** A shared test source is still two separate binaries. `usb_test` on AArch32 reproduced a defect that had already been fixed, purely because only the AArch64 target had been rebuilt; the symptom was indistinguishable from the original bug.
- **A hardware run is one test per power cycle.** It is `load_image` into RAM over the previous test's leftovers, and the Pi has no SRST — the Cortex-A53 debug target has no reset method a script can drive. `run-hw.sh` refuses a suite for this reason.
- **A hardware budget is not the test's own duration.** It is dominated by semihosting traps, which scale with how much a test prints. `smp_test4` reaches its verdict at t=9597 ms of target time and still needs more than 120 s of wall clock.
- **Invalid ACK (0) in DAP response is not a firmware fault.** The debug link gave up; once it is gone nothing services the semihosting traps, so every core halts inside one and it reads like a hang. Drop the JTAG clock with `UOS_HW_ADAPTER_KHZ=1000` and check the supply. The runner reports this as DEBUG LINK LOST.
- **QEMU suites must run one at a time.** Running two four-core suites concurrently, or alongside a build, starves a vCPU: `smp_test2` stalled with three workers finished and the fourth frozen mid-loop. On an idle host the same binary passes 5 out of 5. A stalled worker with no fault is contention, not a bug — re-run before investigating.
- **The AArch32 suite runs on `raspi3b` with the boot shim, never `raspi2b`.** The port is built `-mcpu=cortex-a53`, so its load-acquire instructions are undefined on the raspi2b Cortex-A7 and it faults at the first one. QEMU's raspi3b starts its cores in AArch64, so a 20-line stub drops to AArch32 and jumps to the image. Hardware needs none of this: `config.txt` sets `arm_64bit=0`.
- **rtk's `diff` reports "Files are identical" for files with different MD5s.** Seen on `sd.cpp`, `bcm2837.hpp`, `bcm_irq.hpp`. Use `md5sum` or Python `difflib` for anything that matters.
- **rtk** silently dropped the `-u` flag from `git push -u`; tracking was set with `git branch --set-upstream-to`. Expect this on new branches.
- Do **not** add a blanket `*.html` ignore rule — the kernel ships three doxygen templates as `.html`.
- There are **214 tracked Makefiles**, not the 24 an early count suggested.
- A test app is only ~5 tracked files; the ~100 others per directory are untracked build artifacts.
- **The SMP port contract is small.** The whole kernel patch is 379 lines and asks a port for only: `OS_USE_SMP_SCHEDULER`, `OS_NCPU`, `port_cpu_id`, `port::scheduler::switch_stacks`, `port::stack::element_t`, plus a kernel lock and an IPI.
- **cortexm SMP is not from scratch, and it is not portable.** pico2's `os-core.cpp` is a complete dual-core Cortex-M33 SMP port, but its lock, IPI and CPU index are RP2350 SIO registers. It is 91% the same file as upstream's single-core core, and its port headers already handle every M profile (`6M`, `7M`, `7EM`, `8M_MAIN`) with the SMP parts gated on `OS_USE_SMP_SCHEDULER`. They are **not merged**: the three STM32 boards are hardware-proven on upstream's core, so a board names the core it was proven with (`UOS_BOARD_PORT_CORE`).
- **pico2 forces `-D__ARM_ARCH_7EM__`** although the M33 is ARMv8-M, exactly as the predecessor's Makefiles did. The v7E-M path is the one that board was brought up on; letting the compiler pick `__ARM_ARCH_8M_MAIN__` would change the scheduler under a working board.

- **pico2's `smp-test0` compiles no kernel.** Nine files, none of them `µOS++` — it is the bare-metal dual-core bring-up test that runs before any scheduler exists. `uos_add_app()` always links `micro-os-plus::iii`, so it is parked in `test/pico2/.pending/` rather than rewritten.
- **Flashing the Pico 2: resume cm1 BEFORE cm0.** With `USE_SMP 0` the rp2350 target exposes two targets and `reset init` halts both, while one `resume` resumes only the current one. A debug-halted core 1 will not boot from the PSM reset `launch_core1()` issues, so the launch hangs forever on the bootrom readiness word. Also `reset init`, never a bare `reset` — that does not re-run the bootrom/XIP setup and the new image never boots.
- **The STM32 boards are the workspace's only `OS_NCPU=1` boards.** They are therefore the only ones that build the kernel's non-SMP branch at all.
- **`posix-arch` is pristine upstream v1.0.1**, untracked in the old workspace. Single host thread, `ucontext` coroutines, cooperative only.
- **Three POSIX defects to fix when going multi-threaded:** `sigprocmask` is unspecified in a multithreaded process (use `pthread_sigmask`); `setitimer(ITIMER_REAL)` delivers to an arbitrary thread (use `timer_create` with `SIGEV_THREAD_ID`); `errno` / `thread_local` are host-thread local, so `µOS++` thread migration between CPUs corrupts them.